# Student Academic Manager

Project Proposal for UCS503
Submitted to: Dr. Raghav B. Venkataramaiyer
**Thapar Institute of Engineering and Technology**

Ria Bansal, CSED* Avneet Kaur Bhatia, CSED† Ishaan Garg, CSED‡

September 9, 2026

## 1 Higher-order goal

*. . . secondary framing*

This project aims to improve the organization, accessibility, and effectiveness of academic management for students through a centralized web-based platform. While academic activities are often distributed across different applications, documents, and manual processes, the immediate engineering objective is to integrate essential academic-management functions into a single system that provides students with a structured and convenient learning environment.

The proposed Student Academic Manager (SAM) combines academic record management with learning and self-assessment capabilities. In addition to managing academic information, the system introduces a mock-test feature in which students can upload handwritten answers, extract the written content using Optical Character Recognition (OCR), and obtain an automated evaluation based on the submitted answer.

## 2 Time-to-value

*. . . primary heuristic*

- Develop the core academic-management workflow first, followed by mock tests and handwritten-answer evaluation.
- Build modules such as authentication, academic records, mock tests, OCR, and evaluation independently for faster development and testing.
- Prioritize a functional end-to-end system before adding secondary features and interface refinements.
- Continuously test the frontend, backend, database, and OCR integration to identify and resolve issues early.

## 3 Problem Statement

Students commonly manage their academic activities using multiple disconnected platforms and manual resources. Academic records, study materials, tests, marks, and self-assessment activities may be maintained separately, making it difficult for students to obtain a consolidated view of their academic progress.

Common challenges include:

*Roll No: 1024030621, Email: rbansal1_be24thapar.edu
†Roll No: 1024030619, Email: abhatia2_be24thapar.edu
‡Roll No: 1024030604, Email: igarg1_be24thapar.edu

- **Fragmentation:** Academic information and learning activities are distributed across different platforms, files, and applications.
- **Manual management:** Students may need to maintain academic records, test performance, and other information manually.
- **Limited self-assessment:** Conventional online tests generally require students to enter answers directly into a digital interface and may not reflect handwritten examination practices.
- **Manual evaluation effort:** Handwritten answers require significant effort to review and evaluate, particularly when students want frequent practice.
- **Lack of centralized progress tracking:** Students may not have a single interface through which they can view academic information and monitor their performance across mock tests.
- **Difficulty in digitizing handwritten responses:** Handwritten answers are naturally produced on paper, but converting these responses into machine-readable text for further processing is challenging because handwriting varies considerably between individuals.

### 3.1 Why This Matters Now – Contextual Relevance

With increasing dependence on digital academic tools, there is a need for systems that combine **academic management with active learning and self-assessment**. A centralized platform can reduce repetitive manual work while providing students with a more structured way to monitor their academic activities and practice examination-style questions.

The addition of handwritten-answer processing makes the system particularly relevant to examination preparation, where students frequently write answers on paper rather than typing them.

## 4 Proposed Solution *. . . Software Proposition*

### 4.1 Overview

- **Student authentication:** Secure registration/login and user-specific access to academic information.
- **Academic management:** Centralized storage and presentation of relevant student academic information.
- **Dashboard:** A consolidated interface for accessing academic information, tests, performance, and other system features.
- **Mock-test module:** Students can attempt tests consisting of academic questions.
- **Handwritten-answer submission:** Students can write answers on paper, capture/scan them, and upload the resulting image.
- **OCR processing:** The uploaded handwritten answer is processed to extract machine-readable text.
- **Automated evaluation:** The extracted answer is compared with the expected answer or evaluation criteria to generate an assessment.
- **Performance tracking:** Test results and evaluation outcomes can be stored and presented to students for future reference.

### 4.2 Core workflow

1. The student logs into the **SAM** platform.
2. The student accesses the mock-test module and selects an available test.
3. Questions are displayed along with the required answer format.
4. The student writes the answers on paper.
5. The student uploads an image of the handwritten answer.
6. The system preprocesses the uploaded image and applies OCR to extract the handwritten text.
7. The extracted text is processed and matched against the expected answer/evaluation criteria.
8. The system generates an evaluation based on the answer.
9. The student can view the result, extracted response, and relevant performance information.
10. Test results are stored so that performance can be monitored over subsequent attempts.

### 4.3 Operational constraints for a semester-scale academic system

1. The application will use a responsive web interface so that students can access it from commonly available laptops and smartphones.
2. The OCR component will use existing OCR technologies/APIs or libraries rather than attempting to develop a handwriting-recognition model from scratch.
3. The system will focus on reliable integration of academic management, OCR processing, and evaluation rather than claiming state-of-the-art handwriting recognition.
4. Evaluation results will be treated as automated assistance rather than an absolute replacement for human academic assessment.
5. Uploaded handwritten-answer images will be limited to appropriate formats and sizes to maintain reasonable processing time and storage requirements.

## 5 Solution Approach *... Engineering focus*

This is primarily a software-engineering project. The focus is on developing an integrated academic-management platform and successfully connecting its frontend, backend, database, OCR, and evaluation components into a functional workflow.

### 5.1 Handwritten-answer OCR and evaluation *... AI-assisted component*

The handwritten-answer feature will follow a multi-stage processing pipeline:

1. **Image Upload:** The student uploads a clear image or scanned copy of the handwritten answer.
2. **Image Preprocessing:** The system may perform operations such as resizing, noise reduction, contrast enhancement, or thresholding to improve OCR performance.

3. **Text Extraction:** An OCR engine processes the image and converts the handwriting into machine-readable text.
4. **Text Processing:** The extracted text is cleaned and normalized before evaluation.
5. **Answer Evaluation:** The processed response is compared with the expected answer using suitable text-matching or semantic evaluation techniques.
6. **Result Generation:** The system generates a score or evaluation and displays the result to the student.
7. **Storage:** The relevant test attempt and result are stored in the database for future performance tracking.

Because handwriting recognition can be affected by image quality, handwriting style, and layout, the system will explicitly account for OCR limitations rather than assuming perfect extraction.

### 5.2 Web architecture *... web-based solution*

The application will follow a typical **three-tier web architecture**:

- **Frontend:** Provides the student interface for authentication, academic information, mock tests, answer uploads, and result visualization.
- **Backend:** Handles authentication, API requests, test management, file processing, OCR integration, evaluation logic, and communication with the database.
- **Database:** Stores student information, academic records, test questions, test attempts, extracted answers, and evaluation results.
- **OCR/Evaluation Layer:** Processes uploaded handwritten-answer images and converts them into text before passing the response to the evaluation component.

The separation of these components will make the application easier to test, maintain, and extend.

### 5.3 Testing and integration

Testing will be performed throughout development rather than only after implementation.

The testing process will include:

- **Unit testing:** Individual backend functions and evaluation logic will be tested independently.
- **Integration testing:** Communication between frontend, backend, database, OCR, and evaluation components will be tested.
- **Input validation:** Invalid login details, unsupported files, oversized uploads, and incomplete test submissions will be handled appropriately.
- **OCR validation:** Handwritten samples with different image qualities will be used to identify extraction errors.
- **End-to-end testing:** The complete workflow from mock-test attempt to handwritten-answer upload and evaluation will be tested.
- **User testing:** A small group of students can test the system to identify usability issues and practical limitations.

# 6 Evaluation Criterion *... measurable and attributable*

The effectiveness of SAM will be evaluated using measurable criteria associated with its major workflows.

## 6.1 Primary evaluation metric

**End-to-end mock-test processing success:** the percentage of valid handwritten-answer submissions that successfully pass through image upload, OCR extraction, evaluation, and result generation without system failure.

**Target:** At least **90% successful processing** for valid test submissions under the defined testing conditions.

**Attribution:** The system will log successful and unsuccessful processing attempts at each stage of the workflow.

## 6.2 Secondary evaluation metrics

1. **OCR extraction quality:** Measure the accuracy of extracted text against manually transcribed handwritten samples.
2. **Evaluation consistency:** Compare automated evaluation results with manually evaluated sample answers.
3. **Processing time:** Measure the time required from answer upload to generation of the evaluation result.
4. **System reliability:** Verify that authentication, test submission, file upload, and result retrieval operate correctly during testing.
5. **User usability:** Collect student feedback regarding ease of attempting tests, uploading answers, and understanding evaluation results.
6. **Academic-management accessibility:** Evaluate whether students can access relevant academic information and test results from a centralized interface.

## 6.3 validation plan *... high-level*

1. Prepare a collection of handwritten answer samples with different handwriting styles and image qualities.
2. Process the samples through the OCR pipeline and compare extracted text against manually transcribed text.
3. Conduct mock-test attempts using representative academic questions.
4. Compare automated evaluations with manually evaluated answers for a subset of responses.
5. Measure processing time and identify common OCR/evaluation failure cases.
6. Conduct user testing with a small group of students and collect structured feedback on usability and usefulness.

## 7 Scalability *. . . with foresight*

The system should be designed so that additional students, tests, and academic records can be supported without requiring major architectural changes..

1. **Scalable (theoretically):** The system can support increasing amounts of academic and test data through:

   - Separate frontend and backend components.
   - Database indexing for frequently accessed records.
   - Pagination for academic records, tests, and results.
   - Efficient API-based communication.
   - Modular OCR and evaluation services that can be improved independently.
   - Appropriate file-size and storage management for uploaded answer images.

2. **Operationally deployable (practically):** The application can be deployed using commonly available web-hosting and database services. The architecture will allow the database, backend server, frontend, and file-processing components to be deployed independently when required.

   The OCR component can also be replaced or upgraded without redesigning the complete application, allowing future versions of SAM to use improved handwriting-recognition technologies.

## 8 Engine availability heuristic

A mature set of software technologies is available to implement SAM as a web-based academic-management platform:

- **Frontend framework:** React or equivalent modern web technology.
- **Backend:** Node.js with Express.js for API development.
- **Database:** MySQL for structured academic and user data.
- **Authentication:** Session-based or token-based authentication.
- **File processing:** Server-side handling of uploaded handwritten-answer images.
- **OCR:** An existing OCR engine/library or API capable of processing handwritten text.
- **Evaluation:** Rule-based, text-similarity, or appropriate semantic comparison techniques depending on the type of question.
- **Testing:** Unit and integration testing frameworks.
- **Version control:** Git and GitHub for collaborative development and version management.

Using mature technologies allows the project to focus on **system integration, usability, correctness, and academic utility** rather than developing fundamental infrastructure from scratch.

# 9 Project scope and deliverables *. . . iteration-friendly*

## 9.1 Initial deliverable *. . . first iteration*

1. Student registration and login.
2. Student dashboard.
3. Academic information management and display.
4. Database integration.
5. Basic mock-test creation/access functionality.
6. Student test-attempt workflow.
7. Backend APIs for managing students, tests, and results.
8. Basic validation and error handling.

## 9.2 Subsequent deliverables

1. Handwritten-answer image upload.
2. Image preprocessing for improved OCR.
3. Handwritten text extraction using OCR.
4. Automated answer evaluation.
5. Storage of extracted answers and evaluation results.
6. Performance/result dashboard.
7. Improved test history and filtering.
8. OCR accuracy testing using multiple handwritten samples.
9. User testing and interface refinement.
10. Deployment of the complete system.

# 10 Risks and mitigations

1. **OCR inaccuracies:** Handwriting varies significantly between users and may result in incorrect text extraction. This will be mitigated through image-quality requirements, preprocessing, OCR validation, and testing with multiple handwriting samples.
2. **Poor image quality:** Blurred, poorly lit, or skewed images can reduce OCR accuracy. The system will provide basic upload guidelines and preprocessing where possible.
3. **Evaluation ambiguity:** Some academic answers may have multiple valid formulations. The evaluation component will therefore use suitable comparison methods and clearly define the types of questions supported.
4. **Processing delays:** OCR and evaluation may require more processing time than normal database operations. Processing will be optimized and tested using representative answer images.

5. **Data privacy:** Student academic information and uploaded answer sheets will be treated as application data and access will be restricted to authorized users.

6. **Integration complexity:** Frontend, backend, database, OCR, and evaluation components may produce integration issues. Development will therefore proceed incrementally with module-level and integration testing.

7. **Limited semester time:** Core academic-management and mock-test workflows will be prioritized before optional enhancements.

## 11 Summary

This project proposes Student Academic Manager (SAM), a centralized web-based platform designed to combine academic management with digital self-assessment. The system provides students with a unified interface for managing academic information and attempting mock tests while introducing an additional handwritten-answer workflow.

The key technical feature is the integration of OCR-based handwritten text extraction and automated answer evaluation. Students can write answers on paper, upload their responses, and have the system extract and evaluate the submitted content. The project emphasizes practical software engineering through modular architecture, database integration, API development, OCR integration, testing, and measurable validation.

The proposed system remains feasible within a semester by using existing mature technologies for web development, database management, OCR, and evaluation. Future extensions can include improved handwriting recognition, more advanced semantic evaluation, richer performance analytics, and support for additional academic workflows